

I am doing a Vibe Coding lab on a Python Pygame Q*bert game.

Task 1: Fix the pyramid projection bug.

The README says:

"Each cube's screen position is computed from its (row, col) grid coordinates, and the whole pyramid should come out as a symmetric triangle. In the current build the pyramid is visibly skewed from the very first frame — rows drift sideways in a way that breaks the triangular symmetry. Look at the arithmetic combining row and col in `cube_center`, and check whether the row offset should be using integer division or true division."

The current function is:

```
def cube_center(row, col):
    return pygame.Vector2(
        WIDTH / 2 + (col - row // 2) * CUBE_W,
        90 + row * CUBE_H
    )
```

Explain what the bug is, why it causes the pyramid to become asymmetric, and give me the minimal code change needed to fix it. Do not modify anything else.

The bug is this part:

</> Python

```
col - row // 2
```

`//` is **integer (floor) division**, so odd-numbered rows lose the required half-cube offset.

For example:

- `row = 2 → 2 // 2 = 1` 
- `row = 3 → 3 // 2 = 1`  — should be 1.5
- `row = 4 → 4 // 2 = 2` 

- `row = 5 → 5 // 2 = 2` ❌ — should be `2.5`

Because the pyramid's rows need to shift horizontally by **half a cube for every increase in row**, truncating `.5` on odd rows makes those rows sit too far to one side. The error accumulates visually and the pyramid becomes skewed instead of symmetric.

Minimal fix

Change **only** `//` to `/`:

</> Python

```
def cube_center(row, col):
    return pygame.Vector2(
        WIDTH / 2 + (col - row / 2) * CUBE_W,
        90 + row * CUBE_H
    )
```

That's it. **Do not modify anything else.**

I am continuing my Vibe Coding lab on this Python Pygame Q*bert game.

Task 2: Implement `cube_palette(level)`.

The README says:

"Called once per frame in draw, as `colors = cube_palette(self.level)` or `DEFAULT_PALETTE`. It receives the current level number and should return a list of `TARGET + 1` (currently 3) (r, g, b) colors — one per stage from unpainted to fully painted — or None to keep `DEFAULT_PALETTE`. Idea: return a different 3-color palette for each level."

The current function is:

```
def cube_palette(level):
    pass
```

Please:

1. Explain what this function needs to do.

2. Suggest a simple implementation that gives different 3-color palettes for different levels.
3. Keep the implementation self-contained inside `cube_palette()`.
4. Do not modify any other functions or game logic.
5. Keep the colors as valid RGB tuples with values from 0 to 255.
6. Give me the exact code I should use and explain it briefly.

I want a simple, reliable implementation suitable for a student lab, not an unnecessarily complicated solution.

1. What `cube_palette(level)` needs to do

The function receives the **current game level** and should return either:

- A list of **3 RGB color tuples**, because `TARGET + 1 = 3`.
- Or `None`, which tells the game to use `DEFAULT_PALETTE`.

So, for example, a valid palette is:

`</>` Python

```
[(40, 40, 40), (80, 160, 220), (240, 240, 240)]
```

The three colors represent the cube's stages from **unpainted** → **partially painted** → **fully painted**.

2. Simple implementation

Use a few predefined palettes and cycle through them based on the level:

`</>` Python

```
def cube_palette(level):
    palettes = [
        [(60, 60, 60), (80, 160, 220), (240, 240, 240)],
        [(70, 40, 90), (150, 80, 180), (240, 200, 255)],
        [(50, 80, 50), (90, 180, 100), (220, 255, 220)],
        [(90, 60, 30), (200, 130, 50), (255, 230, 150)]
    ]

    return palettes[(level - 1) % len(palettes)]
```

Why this works

- `palettes` contains **4 different 3-color palettes**.

- Every color is a valid (r, g, b) tuple with values from 0 to 255 .
- level - 1 makes **level 1 use the first palette**.
- % len(palettes) means higher levels simply cycle through the available palettes:
 - Level 1 → palette 1
 - Level 2 → palette 2
 - Level 3 → palette 3
 - Level 4 → palette 4
 - Level 5 → palette 1 again

Everything is contained inside `cube_palette()` , so **nothing else in the game needs to be changed**.

I am continuing my Vibe Coding lab on a Python Pygame Q*bert game.

Task 3: Implement `on_cube_completed(cell)`.

The README says:

"Called from `paint()` the instant a specific cube first reaches its target stage — not on every hop onto it, only the hop that finishes it. It receives the (row, col) cell that was just completed. Its return value is ignored. Idea: a brief flash on that cube, or a small bonus beyond the 25 points already awarded per paint step."

The current function is:

```
def on_cube_completed(cell):
    """Called when a cube first reaches its target colour; add a flash,
    sound, or bonus here."""
    pass
```

Important context:

- The function currently takes only cell as an argument.
- Its return value is ignored.
- Do not change the function signature.
- Do not modify the existing scoring/game logic in other functions unless absolutely necessary.
- The feature should visibly do something when a cube is completed.

- Keep the implementation simple and suitable for a student lab.

Please:

1. Explain what the function should do.
2. Propose the simplest reliable implementation.
3. Give the exact code changes required.
4. Clearly identify if anything outside `on_cube_completed()` must change and why.
5. Do not rewrite unrelated parts of the game.

1. What `on_cube_completed(cell)` should do

`paint()` calls this function **only when a particular cube reaches its final target colour**.

So this function is meant to be a small **event hook**. It can trigger something extra such as:

- a sound,
- a visual notification,
- a small bonus,
- or a completion message.

Since its return value is ignored, it should simply **perform the effect itself**.

2. Simplest reliable implementation

Since we don't want to depend on any sound files, sprites, timers, or changes to the drawing logic, the safest student-lab implementation is a small visible message in the Pygame window title.

Use:

</> Python

```
def on_cube_completed(cell):  
    """Called when a cube first reaches its target colour; add a flash, sound, or bonus here."""  
    pygame.display.set_caption(f"Q*bert - Cube completed: {cell}")
```

For example, completing cube (2, 1) will make the window title show:

</> Plain text

```
Q*bert - Cube completed: (2, 1)
```

3. Exact code change

Replace:

</> Python

```
def on_cube_completed(cell):  
    """Called when a cube first reaches its target colour; add a flash, sound, or bonus here."""  
    pass
```

with:

</> Python

```
def on_cube_completed(cell):  
    """Called when a cube first reaches its target colour; add a flash, sound, or bonus here."""  
    pygame.display.set_caption(f"Q*bert - Cube completed: {cell}")
```

4. Does anything else need to change?

No.

You don't need to change `paint()`, `scoring`, `draw()`, the function signature, or any other game logic.

`paint()` already calls:

</> Python

```
on_cube_completed(cell)
```

at the appropriate moment, so your new function will automatically run **only when the cube is actually completed**, rather than on every hop.

This is deliberately simple: **one function, one line, no new global variables, no assets, and no changes elsewhere.**

I am continuing my Vibe Coding lab on a Python Pygame Q*bert game.

Task 4: Implement `bonus_life_threshold()`.

The README says:

"Called every frame from update(). Return an integer score threshold at which the player earns an extra life, or None to disable bonus lives. The bookkeeping (self.bonus_awarded) is already implemented. Idea: return 1000 so the player earns one bonus life after reaching 1000 points."

The current function is:

```
def bonus_life_threshold():
    """Return a score value at which the player earns an extra life, or
    None to disable bonus lives."""
    pass
```

Important context:

- The function takes no arguments.
- It must return either an integer score threshold or None.
- The existing game logic already handles checking the score and awarding the bonus life.
- Do not modify the existing scoring or bonus-life bookkeeping.
- Keep the implementation simple and suitable for a student lab.

Please:

1. Explain what the function should do.
2. Propose the simplest reliable implementation.
3. Give the exact code change required.
4. Explain why returning 1000 works with the existing game logic.
5. Do not rewrite unrelated parts of the game.

1. What `bonus_life_threshold()` should do

This function simply tells the existing game logic:

"At what score should the player get an extra life?"

It takes **no arguments** and should return either:

- An integer, such as `1000` → enable bonus life at 1000 points.
- `None` → disable bonus lives.

The README specifically suggests `1000`.

2. Simplest implementation

Replace `pass` with `return 1000`.

3. Exact code change

</> Python

```
def bonus_life_threshold():  
    """Return a score value at which the player earns an extra life, or None to disable bonus lives."""  
    return 1000
```

4. Why 1000 works

The existing `update()` logic already checks the value returned by this function against the player's score and uses `self.bonus_awarded` to make sure the bonus isn't repeatedly awarded.

So when the player's score reaches **1000 points**, the existing bookkeeping awards the extra life.

You **do not need to add any scoring or life logic** here.

5. Anything else to change?

Nothing else.

Only change:

</> Python

```
pass
```

to:

</> Python

```
return 1000
```

This is the complete implementation for Task 4.